00044

Student ID:

Machine No:

Sri Lanka Institute of Information Technology

B.Sc. Honours Degree in Information Technology

Specialized in Information Technology

Final Examination (Computer Based)
Year 2, Semester 1 (2024)

IE2021 – Object Oriented Programming

Duration: 3 Hours

June 2024

Instructions to Candidates:

- This paper contains four questions. Answer All Questions.
- Marks for each question are given on the paper. Total Marks: 100.
- Create a folder on the Desktop with your student registration number and store all your program files inside that.
- Create a separate Project for each question. The name of the project is provided in the question.
- This paper contains 06 pages including the Cover Page.

Instructions to Candidates when submitting:

- Save all your work.
- Delete all unnecessary files from each project folder (There should be 4 folders for 4 projects named as Question01, Question02, and Question03 inside your ID folder, and each folder should contain the source code (.java files)) only.
- Zip the Student ID folder (Zipped folder also should be named with Student ID number).
- Upload the zipped folder to the correct link.

Question 1 (25 marks)

This question is based on the **OOP concepts**. Implement the necessary classes, along with their required attributes and methods, based on the following description.

You are required to create a library catalog system which consists of two classes, **Book** and **LibraryCatalog**.

The **Book** class should have a bookID (int), title (String), author (String), genre (String), publicationYear (int), availableCopies (int). The class should include a default constructor and an overloaded constructor to initialize these attributes. Additionally, the following methods need to be implemented:

- **decreaseAvailableCopies()**: This method should decrease the number of available copies of the book by 1 whenever a book is borrowed.
- **increaseAvailableCopies()**: This method should increase the number of available copies of the book by 1 whenever a book is returned.
- **displayBookDetails()**: This method should display all the details of the book.

The **LibraryCatalog** class should have an arraylist called catalog to store a collection of book objects in the library. The class should include a default constructor and the following methods:

- **addBook(Book book)**: This method should add a book to the library catalog.
- **searchBook(int bookId)**: This should search for a book by its id and display its details.
- **borrowBook(int bookId)**: This method should borrow a book from the library. It should decrease the available copies of the book if it is available.
- **returnBook(int bookId)**: This method should simulate returning a book to the library. It should increase the available copies of the book.

Implement a class called "**LibraryApp**" with the main method. In the main method, create an instance of the **LibraryCatalog** class. Populate the library catalog with at least 5 books using the **addBook()** method. Perform the following operations:

- Search for a book by title using the **searchBook()** method.
- Borrow a book using the **borrowBook()** method and return a book using the **returnBook()** method.
- Display the details of all the books in the library catalog using the **displayBookDetails()**

Save the project as **Question01**.

Question 2 (30 marks)

This question is based on the **Collection Framework** and **Generics**.

a) Develop a Java program for managing a student record system using **HashMap**. The program should perform the following tasks:

i). Create an empty HashMap to store student records. Then prompt the user to enter the details of students one by one. Accept inputs for student ID (int) and name (String) as keyboard inputs. Keep accepting inputs until the user enters "-99". Add each entered student record to the HashMap. (04 marks)

ii). Once the user finishes entering the student records (once the user enters "-99"), ask the user to enter the id of the student that needs to be removed. If the student's id is found in the HashMap, remove the corresponding student record. Display a message indicating whether the record was successfully removed or not in the case if the student ID was not found. (06 marks)

*[Hint: Use **containsKey(id)** to find a key in the HashMap]*

iii). Find and display the total number of students in the HashMap. (02 marks)

iv). Print all the remaining student records in the HashMap. (03 marks)

Save the project as **Question02A**.

b) Implement a Java program to demonstrate the use of **generics** in a class called "**GenericInventory**". The program should accomplish the following tasks:

i). Define a generic method called "**calculateMaximum()**" that takes an array of a numeric data type (Integer, Double, Float) as input and displays the maximum value from the array. (05 marks)

*[Hint: convert the array elements to double values using **doubleValue()** method]*

ii). Define another generic method called "**calculateProduct()**" that takes an array of a numeric data type (Integer, Double, Float) as input and returns the product of its elements. This should multiply all the elements in the array and return the result. (05 marks)

iii). Write another class called "**InventoryManager**" which contains the main method. Create two objects from the "**GenericInventory**" class, where the first object accepts an Integer type array for the above two methods, and the second object accepts a Double type array for the above two methods. Display the results using proper printing statements. (05 marks)

Save the project as **Question02B**.

Question 3 (20 marks)

This question is based on the **Exception Handling**.

a) Write a program that allows warehouse manager to enter the quantity of items in an order. The program should perform three validations using different custom exception classes:

i). **InsufficientQuantity**, is a custom exception class. If the entered quantity is less than 1, it should throw this exception and print the error message as "Insufficient quantity. Please enter a quantity greater than 0.". (06 marks)

ii). **QuantityOutOfRange** is another exception class. If the entered quantity is not within the valid range of 1 to 1000, it should throw this exception and print the error message as "Quantity out of range. Quantity must be between 1 and 1000.". (06 marks)

iii). **QuantityNotNumeric** is the third exception class. If the entered quantity is not a numeric value, it should throw this exception and print the error message: "Invalid input. Please enter a numeric quantity.". (06 marks)

b) Write another class called **OrderProcessor** to get the user inputs for order quantity and manage order processing. Then validate those for the above three conditions. If the input has one or more custom exceptions, you should have a proper try catch statements to handle the exceptions. (07 marks)

Hint: Refer to the sample output given below

Save the project as **Question03**

Sample Outputs:

```
Enter the quantity of items in the order: 2000
Quantity out of range. Quantity must be between 1 and 1000.

Enter the quantity of items in the order: AB
Invalid input. Please enter a numeric quantity.


Enter the quantity of items in the order: -2
Insufficient quantity. Please enter a quantity greater than 0.

Enter the quantity of items in the order: 80
Order processing successful for 80 items.
```

Question 4 (25 marks)

This question is based on the **Design Patterns** implementation.

You are required to implement a multimedia control system using the **Command Pattern** and the **Singleton Pattern**. The system consists of two receiver classes, namely **Television** and **SoundSystem**, each with specific functionalities. The Command Pattern is implemented with a Command interface and several concrete command classes to control the device operations. A class called **RemoteControl** is utilized, accommodating the singleton pattern to manage these commands effectively.

In the **Television** class, three methods are implemented: "turnOn()", "turnOff()", and "setChannel(int channel)", each with appropriate display statements indicating their functionality. Similarly, in the SoundSystem class, three methods are implemented: "turnOn()", "turnOff()", and "increaseVolume()", with corresponding display statements.

For the command design pattern implementation, a method named "execute()" is defined within the **Command** interface. This interface is then implemented in various concrete command classes such as **TvOnCommand**, **TvOffCommand**, **SetChannelCommand**, **SoundOnCommand**, **SoundOffCommand**, and **IncreaseVolumeCommand**, each representing a specific command operation.

Regarding the Singleton pattern, within the **RemoteControl** class, an array of Command objects is defined to store the commands assigned to each button on the remote. A method called "setCommand(int slot, Command command)" is implemented to assign a command to a specific button slot in the remote control. Additionally, a method named "pressButton(int slot)" is created to simulate pressing a button on the remote control, checking if a command is assigned to the specified button slot and executing it by calling its "execute()" method.

Draw a class diagram showcasing the relationships and interactions between the classes, interfaces, and methods mentioned above. Use the given space on the paper to draw the diagram.